IME 775 — Lecture 16

Training Neural Networks: Loss Functions, Gradient Descent, and Backpropagation

---

1. The Training Problem

In Chapter 7 we **manually** chose weights to make neural networks compute specific functions (logic gates). In practice, we have a **training set** of input-output pairs $\{(\vec{x}_i, \bar{y}_i)\}_{i=1}^{N}$ and we want to **automatically** find weights that make the network's output $\vec{y}_i = \text{MLP}(\vec{x}_i)$ match the target $\bar{y}_i$ for all examples.

This requires three ingredients:

1. A **loss function** that measures how wrong the network is
2. **Gradient descent** to adjust weights in the direction that reduces loss
3. **Backpropagation** to efficiently compute the gradients

---

2. Mean Squared Error (MSE) Loss

Definition

The standard loss function for regression tasks:

$$L = \frac{1}{2}\sum_{i=1}^{N} \|\bar{y}_i - y_i\|^2 = \frac{1}{2}\sum_{i=1}^{N}\sum_{j}(\bar{y}_{ij} - y_{ij})^2$$

where:

- $\bar{y}_i$ is the **target** output for training example i
- $y_i = \text{MLP}(\vec{x}_i)$ is the **predicted** output for input $\vec{x}_i$
- The factor $\frac{1}{2}$ is a convenience: it cancels the exponent 2 when we take the derivative

Properties

- $L \geq 0$ always
- $L = 0$ if and only if $y_i = \bar{y}_i$ for all i (perfect predictions)
- L depends on the weights and biases **through** $y_i = \text{MLP}(\vec{x}_i)$

Single Example Loss

For one training example:

$$\ell_i = \frac{1}{2}\|\bar{y}_i - y_i\|^2$$

The total loss is $L = \sum_{i=1}^{N} \ell_i$.

Workout: A network with one output produces predictions $y_1 = 0.7$, $y_2 = 0.3$ for targets $\bar{y}_1 = 1.0$, $\bar{y}_2 = 0.0$. Compute the MSE loss.

Solution:

$$L = \frac{1}{2}(1.0 - 0.7)^2 + \frac{1}{2}(0.0 - 0.3)^2 = \frac{1}{2}(0.09) + \frac{1}{2}(0.09) = 0.045 + 0.045 = 0.09$$

---

3. The Loss Surface

Intuition

The loss L is a function of **all weights and biases** in the network. For a network with P total parameters, L is a function $\mathbb{R}^P \to \mathbb{R}$. Think of it as a landscape (surface) in $(P+1)$-dimensional space.

Training = finding the **lowest valley** on this surface.

Key features of the loss surface

- **Global minimum**: the lowest point (optimal parameters)
- **Local minima**: valleys that are not the deepest (suboptimal parameters)
- **Saddle points**: points that are a minimum in some directions but a maximum in others
- **Plateaus**: flat regions where the gradient is near zero (common in deep networks)

For neural networks, the loss surface is:

- **Non-convex**: multiple local minima exist
- **High-dimensional**: impossible to visualize directly (P can be millions)
- **Often well-behaved in practice**: despite non-convexity, gradient descent usually finds good solutions

---

4. Gradient Descent

The Core Idea

We want to decrease L by adjusting each weight w. The gradient $\nabla_w L$ tells us the direction of **steepest increase**. So we move in the **opposite** direction:

$$w \leftarrow w - r \cdot \frac{\partial L}{\partial w}$$

where $r > 0$ is the **learning rate**.

Why the Gradient?

From the first-order Taylor expansion:

$$L(w + \Delta w) \approx L(w) + \nabla_w L \cdot \Delta w$$

To decrease L as much as possible for a step of fixed size $||\Delta w|| = \epsilon$, we choose:

$$\Delta w = -\epsilon \cdot \frac{\nabla_w L}{\|\nabla_w L\|}$$

This makes $\nabla_w L \cdot \Delta w = -\epsilon \|\nabla_w L\|$, which is the **most negative** possible dot product. The gradient is provably the steepest descent direction.

For All Parameters Simultaneously

$$W \leftarrow W - r \cdot \frac{\partial L}{\partial W}, \qquad \vec{b} \leftarrow \vec{b} - r \cdot \frac{\partial L}{\partial \vec{b}}$$

Every weight and bias in every layer is updated in the same way.

Workout: Consider a 2 → 2 → 1 network with the following current parameters and computed gradients (after a backward pass). Apply one gradient descent step with learning rate $r = 0.1$.

Layer 0:

$$W^{(0)} = \begin{pmatrix} 0.5 & 0.3 \\ -0.2 & 0.8 \end{pmatrix}, \quad \vec{b}^{(0)} = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$$

$$\frac{\partial L}{\partial W^{(0)}} = \begin{pmatrix} 0.04 & -0.02 \\ 0.06 & 0.01 \end{pmatrix}, \quad \frac{\partial L}{\partial \vec{b}^{(0)}} = \begin{pmatrix} 0.05 \\ -0.03 \end{pmatrix}$$

Layer 1:

$$\vec{w}^{(1)} = \begin{pmatrix} 0.4 & 0.6 \end{pmatrix}, \quad b^{(1)} = 0.2$$

$$\frac{\partial L}{\partial \vec{w}^{(1)}} = \begin{pmatrix} -0.08 & -0.05 \end{pmatrix}, \quad \frac{\partial L}{\partial b^{(1)}} = -0.10$$

Solution:

All six parameter groups are updated simultaneously using $\theta \leftarrow \theta - r \cdot \frac{\partial L}{\partial \theta}$:

$$W^{(0)} \leftarrow \begin{pmatrix} 0.5 & 0.3 \\ -0.2 & 0.8 \end{pmatrix} - 0.1 \begin{pmatrix} 0.04 & -0.02 \\ 0.06 & 0.01 \end{pmatrix} = \begin{pmatrix} 0.496 & 0.302 \\ -0.206 & 0.799 \end{pmatrix}$$

$$\vec{b}^{(0)} \leftarrow \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} - 0.1 \begin{pmatrix} 0.05 \\ -0.03 \end{pmatrix} = \begin{pmatrix} 0.095 \\ -0.097 \end{pmatrix}$$

$$\vec{w}^{(1)} \leftarrow \begin{pmatrix} 0.4 & 0.6 \end{pmatrix} - 0.1 \begin{pmatrix} -0.08 & -0.05 \end{pmatrix} = \begin{pmatrix} 0.408 & 0.605 \end{pmatrix}$$

$$b^{(1)} \leftarrow 0.2 - 0.1 \times (-0.10) = 0.210$$

Notice that every parameter moves in the direction that reduces L: parameters with positive gradients decrease, and parameters with negative gradients increase.

The Learning Rate r

The learning rate is the single most important hyperparameter in neural network training.

r **too large**	r **too small**
Oscillates around minimum	Convergence is extremely slow
Can diverge (loss increases)	Gets stuck in shallow local minima
Overshoots good solutions	Wastes computational resources

Nuanced point — choosing the learning rate: There is no universally correct learning rate. In practice, one often starts with $r = 0.01$ or $r = 0.001$ and adjusts based on whether the loss is decreasing steadily. Modern methods use **adaptive learning rates** (Adam, RMSProp) that adjust r per parameter during training.

Workout: A weight $w = 0.5$ has gradient $\frac{\partial L}{\partial w} = -0.3$. With learning rate $r = 0.1$, compute the updated weight.

Solution:

$$w_{\text{new}} = 0.5 - 0.1 \times (-0.3) = 0.5 + 0.03 = 0.53$$

The negative gradient indicates that increasing w will decrease the loss, so the update moves w from 0.5 to 0.53.

---

5. Backpropagation — Simple Network

Setup

Consider the simplest deep network: **one neuron per layer** with $L+1$ layers (layers 0 through L), sigmoid activation, scalar inputs and outputs.

Each layer l has:

- One weight $w^{(l)}$ and one bias $b^{(l)}$
- Pre-activation: $z^{(l)} = w^{(l)} a^{(l-1)} + b^{(l)}$
- Activation: $a^{(l)} = \sigma\big(z^{(l)}\big)$
- The input to the network is x, so the input to layer 0 is $a^{(-1)} = x$
- The output is $y = a^{(L)}$

The Challenge

We need $\frac{\partial L}{\partial w^{(l)}}$ and $\frac{\partial L}{\partial b^{(l)}}$ for every layer l. The loss depends on $w^{(l)}$ through a long chain of compositions:

$$w^{(l)} \to z^{(l)} \to a^{(l)} \to z^{(l+1)} \to a^{(l+1)} \to \cdots \to a^{(L)} = y \to L$$

The Auxiliary Variable δ

Define:

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}$$

This measures how sensitive the loss is to the pre-activation at layer l. Once we know $\delta^{(l)}$, the weight and bias gradients follow immediately:

$$\frac{\partial L}{\partial w^{(l)}} = \delta^{(l)} \cdot a^{(l-1)}$$

$$\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}$$

Why these gradients?

For a single neuron, the layer only influences the loss through its **pre-activation**

$$z^{(l)} = w^{(l)} a^{(l-1)} + b^{(l)}.$$

Using the chain rule:

$$\frac{\partial L}{\partial w^{(l)}} = \frac{\partial L}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial w^{(l)}}.$$

Since $a^{(l-1)}$ is treated as a constant when differentiating w.r.t. $w^{(l)}$,

$$\frac{\partial z^{(l)}}{\partial w^{(l)}} = a^{(l-1)}.$$

So,

$$\frac{\partial L}{\partial w^{(l)}} = \delta^{(l)} \cdot a^{(l-1)}.$$

Similarly,

$$\frac{\partial L}{\partial b^{(l)}} = \frac{\partial L}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial b^{(l)}}.$$

But

$$\frac{\partial z^{(l)}}{\partial b^{(l)}} = 1,$$

so

$$\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}.$$

Computing δ — From Output to Input

Last layer ($l = L$):

For single-example MSE loss $\ell = \frac{1}{2}(\bar{y} - y)^2$:

$$\delta^{(L)} = \frac{\partial \ell}{\partial z^{(L)}} = -(\bar{y} - y) \cdot \sigma'(z^{(L)})$$

Derivation (last layer):

At the output layer, $y = a^{(L)} = \sigma(z^{(L)})$, so

$$\frac{\partial \ell}{\partial z^{(L)}} = \frac{\partial \ell}{\partial y} \cdot \frac{\partial y}{\partial z^{(L)}}.$$

For $\ell = \frac{1}{2}(\bar{y} - y)^2$,

$$\frac{\partial \ell}{\partial y} = -(\bar{y} - y).$$

Also,

$$\frac{\partial y}{\partial z^{(L)}} = \sigma'(z^{(L)}).$$

Multiplying gives

$$\delta^{(L)} = \frac{\partial \ell}{\partial z^{(L)}} = -(\bar{y} - y)\sigma'(z^{(L)}).$$

Recursion (from layer $l+1$ back to layer l):

$$\delta^{(l)} = \delta^{(l+1)} \cdot w^{(l+1)} \cdot \sigma'(z^{(l)})$$

Derivation (recursive step):

For an internal layer l, the loss depends on $z^{(l)}$ through the next layer:

$$z^{(l)} \to a^{(l)} \to z^{(l+1)} \to \cdots \to \ell.$$

Using chain rule through this path:

$$\frac{\partial \ell}{\partial z^{(l)}} = \frac{\partial \ell}{\partial z^{(l+1)}} \cdot \frac{\partial z^{(l+1)}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}}.$$

Now substitute each factor:

- $\frac{\partial \ell}{\partial z^{(l+1)}} = \delta^{(l+1)}$
- $z^{(l+1)} = w^{(l+1)} a^{(l)} + b^{(l+1)} \Rightarrow \frac{\partial z^{(l+1)}}{\partial a^{(l)}} = w^{(l+1)}$
- $a^{(l)} = \sigma\big(z^{(l)}\big) \Rightarrow \frac{\partial a^{(l)}}{\partial z^{(l)}} = \sigma'\big(z^{(l)}\big)$

Therefore:

$$\delta^{(l)} = \delta^{(l+1)} \cdot w^{(l+1)} \cdot \sigma'\big(z^{(l)}\big).$$

This recursion is the heart of backpropagation: the delta at layer l is computed from the delta at layer $l+1$, propagated **backward** through the network.

Interpretation:

1. $\delta^{(l+1)}$ is the error signal arriving from the next layer.
2. Multiplying by $w^{(l+1)}$ tells how strongly neuron l influences that next-layer error.
3. Multiplying by $\sigma'\big(z^{(l)}\big)$ applies local sensitivity at layer l (if the neuron is saturated, this term is small, so little error flows back).

How Backpropagation and Gradient Descent Work Together

These are two different steps in one training iteration:

1. **Forward pass:** compute predictions and loss.
2. **Backpropagation:** compute all gradients $\frac{\partial L}{\partial W^{(l)}}$, $\frac{\partial L}{\partial \vec{b}^{(l)}}$ using the chain rule.
3. **Gradient descent:** update parameters using those gradients:

$$W^{(l)} \leftarrow W^{(l)} - r\frac{\partial L}{\partial W^{(l)}}, \qquad \vec{b}^{(l)} \leftarrow \vec{b}^{(l)} - r\frac{\partial L}{\partial \vec{b}^{(l)}}.$$

In short: **backpropagation computes the direction; gradient descent takes the step**.

Why "Backpropagation"?

We propagate the **error signal** (δ) from the output layer backward through the network, layer by layer. At each layer, the error is:

1. **Scaled** by the weight connecting to the next layer ($w^{(l+1)}$)

2. **Modulated** by the local derivative ($\sigma'(z^{(l)})$)

This is a direct application of the **chain rule** from calculus.

---

6. Backpropagation — General Network

Extension to Multiple Neurons per Layer

For layer l with multiple neurons, δ becomes a **vector**:

$$\vec{\delta}^{(l)} = \frac{\partial L}{\partial \vec{z}^{(l)}}$$

Last layer ($l = L$):

$$\vec{\delta}^{(L)} = -(\vec{\bar{y}} - \vec{y}) \odot f'(\vec{z}^{(L)})$$

where $\odot$ denotes the **Hadamard (elementwise) product**.

Recursion:

$$\vec{\delta}^{(l)} = \left[(W^{(l+1)})^T \vec{\delta}^{(l+1)}\right] \odot f'(\vec{z}^{(l)})$$

The weight matrix transpose $(W^{(l+1)})^T$ distributes the error from the next layer back to the current layer. Each neuron in layer l receives a weighted sum of the deltas from all neurons in layer $l+1$.

Weight and Bias Gradients

$$\frac{\partial L}{\partial W^{(l)}} = \vec{\delta}^{(l)} (\vec{a}^{(l-1)})^T$$

$$\frac{\partial L}{\partial \vec{b}^{(l)}} = \vec{\delta}^{(l)}$$

The weight gradient is an **outer product** of the delta vector with the previous layer's activation vector.

Dimensions Check

If layer l has n neurons and layer $l-1$ has m neurons:

- $\vec{\delta}^{(l)}: n \times 1$
- $(\vec{a}^{(l-1)})^T: 1 \times m$
- $\frac{\partial L}{\partial W^{(l)}} = \vec{\delta}^{(l)}(\vec{a}^{(l-1)})^T: n \times m$ (same shape as $W^{(l)}$ ✓)

Workout: Complete forward and backward pass on a 1 → 2 → 1 network.

Given:

$$W^{(0)} = \begin{pmatrix} 0.5 \\ -0.3 \end{pmatrix}, \quad \vec{b}^{(0)} = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}, \quad \vec{w}^{(1)} = \begin{pmatrix} 0.4 & 0.6 \end{pmatrix}, \quad b^{(1)} = 0.2$$

Input $x = 1.0$, target $\bar{y} = 1.0$, sigmoid activation.

Solution:

Forward pass:

$$\vec{z}^{(0)} = \begin{pmatrix} 0.5 \\ -0.3 \end{pmatrix}(1.0) + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} 0.6 \\ -0.4 \end{pmatrix}$$

$$\vec{a}^{(0)} = \sigma(\vec{z}^{(0)}) = \begin{pmatrix} \sigma(0.6) \\ \sigma(-0.4) \end{pmatrix} \approx \begin{pmatrix} 0.646 \\ 0.401 \end{pmatrix}$$

$$z^{(1)} = 0.4(0.646) + 0.6(0.401) + 0.2 = 0.258 + 0.241 + 0.2 = 0.699$$

$$y = a^{(1)} = \sigma(0.699) \approx 0.668$$

Loss: $\ell = \frac{1}{2}(1.0 - 0.668)^2 = \frac{1}{2}(0.110) = 0.055$

Backward pass:

$$\delta^{(1)} = -(1.0 - 0.668) \cdot \sigma'(0.699) = -0.332 \times 0.668 \times (1 - 0.668) = -0.332 \times 0.222 = -0.073$$

Gradients for layer 1:

$$\frac{\partial \ell}{\partial \vec{w}^{(1)}} = \delta^{(1)} \cdot (\vec{a}^{(0)})^T = -0.0737 \times \begin{pmatrix} 0.646 & 0.401 \end{pmatrix} = \begin{pmatrix} -0.0476 & -0.0296 \end{pmatrix}$$

$$\frac{\partial \ell}{\partial b^{(1)}} = \delta^{(1)} = -0.0737$$

Backpropagate to layer 0:

$$\vec{\delta}^{(0)} = \left[(W^{(1)})^T \delta^{(1)} \right] \odot \sigma'(\vec{z}^{(0)}) = \begin{pmatrix} 0.4 \\ 0.6 \end{pmatrix} (-0.0737) \odot \begin{pmatrix} 0.646 \times 0.354 \\ 0.401 \times 0.599 \end{pmatrix}$$

$$= \begin{pmatrix} -0.0295 \\ -0.0442 \end{pmatrix} \odot \begin{pmatrix} 0.229 \\ 0.240 \end{pmatrix} = \begin{pmatrix} -0.00675 \\ -0.01061 \end{pmatrix}$$

Gradients for layer 0:

$$\frac{\partial \ell}{\partial W^{(0)}} = \vec{\delta}^{(0)} \cdot x = \begin{pmatrix} -0.00675 \\ -0.01061 \end{pmatrix} \times 1.0 = \begin{pmatrix} -0.00675 \\ -0.01061 \end{pmatrix}$$

$$\frac{\partial \ell}{\partial \vec{b}^{(0)}} = \vec{\delta}^{(0)} = \begin{pmatrix} -0.00675 \\ -0.01061 \end{pmatrix}$$

Gradient descent update (one step, with $r = 0.1$):

$$\vec{w}^{(1)}_{\text{new}} = \vec{w}^{(1)} - r \frac{\partial \ell}{\partial \vec{w}^{(1)}} = \begin{pmatrix} 0.4 & 0.6 \end{pmatrix} - 0.1 \begin{pmatrix} -0.0476 & -0.0296 \end{pmatrix} = \begin{pmatrix} 0.40476 & 0.60296 \end{pmatrix}$$

$$b^{(1)}_{\text{new}} = b^{(1)} - r \frac{\partial \ell}{\partial b^{(1)}} = 0.2 - 0.1(-0.0737) = 0.20737$$

$$W^{(0)}_{\text{new}} = W^{(0)} - r \frac{\partial \ell}{\partial W^{(0)}} = \begin{pmatrix} 0.5 \\ -0.3 \end{pmatrix} - 0.1 \begin{pmatrix} -0.00675 \\ -0.01061 \end{pmatrix} = \begin{pmatrix} 0.500675 \\ -0.298939 \end{pmatrix}$$

$$\vec{b}^{(0)}_{\text{new}} = \vec{b}^{(0)} - r\frac{\partial \ell}{\partial \vec{b}^{(0)}} = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} - 0.1 \begin{pmatrix} -0.00675 \\ -0.01061 \end{pmatrix} = \begin{pmatrix} 0.100675 \\ -0.098939 \end{pmatrix}$$

All gradients are negative, meaning the loss decreases if we increase these weights — consistent with the network under-predicting (0.668 vs target 1.0).

---

7. The Complete Training Algorithm

Algorithm 8.1: Gradient Descent Training

```
Initialize all weights W^(l) and biases b^(l) randomly

Repeat until convergence:
    total_loss = 0
    total_grad_W = 0, total_grad_b = 0   (for each layer)

    For each training example (x_i, y_bar_i):
        1. FORWARD PASS: Compute all z^(l) and a^(l) for l = 0,
..., L
           - a^(-1) = x_i
           - z^(l) = W^(l) * a^(l-1) + b^(l)
           - a^(l) = f(z^(l))
           - y_i = a^(L)

        2. COMPUTE LOSS: ell_i = (1/2) ||y_bar_i - y_i||^2

        3. BACKWARD PASS: Compute all delta^(l) for l = L, L-1,
..., 0
           - delta^(L) = -(y_bar_i - y_i) ⊙ f'(z^(L))
           - delta^(l) = [W^(l+1)^T * delta^(l+1)] ⊙ f'(z^(l))

        4. ACCUMULATE GRADIENTS:
           - total_grad_W^(l) += delta^(l) * (a^(l-1))^T
           - total_grad_b^(l) += delta^(l)

    5. UPDATE PARAMETERS:
       - W^(l) = W^(l) - r * total_grad_W^(l)
       - b^(l) = b^(l) - r * total_grad_b^(l)
```

Key Observations

1. Gradients are **accumulated** over all training examples before updating
2. Each **epoch** = one complete pass through all training examples
3. The forward pass must be completed **before** the backward pass (we need all $\vec{z}^{(l)}$ and $\vec{a}^{(l)}$)
4. Weight initialization matters: all zeros → all neurons learn the same thing. Random initialization breaks this symmetry.

Nuanced: The Vanishing Gradient Problem

During backpropagation, each layer multiplies by:

- $\sigma'(z^{(l)}) \leq 0.25$ (for sigmoid)
- $w^{(l+1)}$ (weights, typically $|w| < 1$ initially)

After L layers, the gradient at the first layer is proportional to:

$$\prod_{l=0}^{L-1} |w^{(l+1)}| \cdot \sigma'(z^{(l)})$$

If each factor is < 1 (as is typical with sigmoid), this product **shrinks exponentially** with depth L. Result: early layers barely learn, making deep networks very hard to train with sigmoid.

Solutions (covered later):

- ReLU activation (does not saturate for positive inputs)
- Batch normalization
- Residual connections (skip connections)

---

8. PyTorch Training Loop

Complete Training Example

```
import torch
import torch.nn as nn

# Define the network: 2 → 4 → 1 with sigmoid activations
model = nn.Sequential(
    nn.Linear(2, 4),
    nn.Sigmoid(),
    nn.Linear(4, 1),
    nn.Sigmoid()
)

# Training data: XOR problem
X = torch.tensor([[0,0],[0,1],[1,0],[1,1]], dtype=torch.float32)
Y = torch.tensor([[0],[1],[1],[0]], dtype=torch.float32)

# Loss function and optimizer
loss_fn = nn.MSELoss()              # MSE loss (includes the 1/N
averaging)
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)  #
Gradient descent

# Training loop
for epoch in range(10000):
    # Forward pass
    y_pred = model(X)

    # Compute loss
    loss = loss_fn(y_pred, Y)

    # Backward pass (computes all gradients via backpropagation)
    optimizer.zero_grad()     # Reset gradients to zero
    loss.backward()           # Backpropagation

    # Update parameters (gradient descent step)
    optimizer.step()

    if epoch % 2000 == 0:
        print(f"Epoch {epoch:5d} | Loss: {loss.item():.6f}")

# Test the trained network
with torch.no_grad():
```

```
    predictions = model(X)
    print("\nPredictions:")
    for i in range(4):
        print(f"  {X[i].tolist()} → {predictions[i].item():.4f}
 (target: {Y[i].item():.0f})")
```

Mapping to the Algorithm

Algorithm Step	PyTorch
Forward pass	`y_pred = model(X)`
Compute loss	`loss = loss_fn(y_pred, Y)`
Reset gradients	`optimizer.zero_grad()`
Backward pass	`loss.backward()`
Update parameters	`optimizer.step()`

Why `zero_grad()`?

PyTorch **accumulates** gradients by default (adds new gradients to existing ones). This is useful for some advanced techniques, but for standard training we must reset to zero before each backward pass. Forgetting `zero_grad()` is one of the most common PyTorch bugs.

Key Takeaways

1. **MSE loss** $L = \frac{1}{2}\sum \|\bar{y} - y\|^2$ measures prediction error; the factor $\frac{1}{2}$ simplifies derivatives
2. The **loss surface** is a high-dimensional landscape; training seeks low valleys
3. **Gradient descent** $w \leftarrow w - r \cdot \nabla_w L$ moves parameters downhill on the loss surface
4. **Backpropagation** efficiently computes all gradients by propagating δ values backward layer by layer
5. The delta recursion: $\vec{\delta}^{(l)} = \left[(W^{(l+1)})^T \vec{\delta}^{(l+1)}\right] \odot f'(\vec{z}^{(l)})$
6. Weight gradients: $\frac{\partial L}{\partial W^{(l)}} = \vec{\delta}^{(l)} (\vec{a}^{(l-1)})^T$ — an outer product of delta and activation

7. **Vanishing gradients** with sigmoid: $\sigma'(x) \leq 0.25$ → gradients shrink exponentially with depth
8. PyTorch automates forward pass, backpropagation, and gradient descent with just a few lines